

Appendix B — The Annotated Dependency Manifest

Every third-party package the node depends on — what it is, why Hathor uses it, and where the book covers it — read straight from `pyproject.toml`.

SUBJECT hathor-core · Appendix B

CHAPTER B · Appendices

BRANCH study/hathor-core-studies

EDITION 2026 · v1

`pyproject.toml` · Runtime deps · Dev deps · Version constraints · Optional extras · Build backend · Per-dependency rationale

Table of Contents

Appendix B — The Annotated Dependency Manifest	3
B.1 The Python platform	3
B.2 Runtime core frameworks	4
B.3 Storage & serialization	5
B.4 Networking & APIs	5
B.5 Cryptography	6
B.6 Observability & logging	7
B.7 CLI, process & utilities	8
B.8 Development & quality toolchain	9
B.9 Build & packaging	10
Recap	11

Appendix B — The Annotated Dependency Manifest

Chapter 13 explained how Poetry declares dependencies — the split between the flexible ranges in `pyproject.toml` and the exact pins in `poetry.lock`, the `~` and `^` operators, the runtime-vs-development groups. This appendix is the payoff: a line-by-line annotation of the actual manifest, so that every package name you have seen flash past in an import or a stack trace has a one-paragraph answer to "what is this, and why is it here?"

Every entry below is read directly from `pyproject.toml` at the repository root. The **version** column is the constraint as declared (not the resolved pin — that lives in `poetry.lock`). The **chapter** column points to where the book treats the package in depth; a dash means the package is a supporting actor with no dedicated treatment. A reminder of the constraint syntax from Chapter 13: `~X.Y.Z` accepts patch updates within `X.Y`; `^X.Y` accepts any release below the next major; `=X` pins exactly; `{git = ...}` pulls from a repository rather than PyPI.

Why pin versions at all? A node must behave identically across every operator's machine, or consensus fractures (Chapter 22). Loose, unpinned dependencies would let two nodes resolve different library versions and, in the worst case, disagree on a validation result. The manifest's discipline — declare ranges, lock exact versions, commit the lock — is part of how the network stays in agreement. → full treatment in **Chapter 13**.

B.1 The Python platform

Package	Version	What it is / why Hathor uses it	Chapter
python	<code>>=3.11,<4</code>	The interpreter constraint itself (<code>pyproject.toml: 56</code>). Poetry refuses to install into an incompatible interpreter, turning "needs Python 3.11+" from a README note into a machine-checked guarantee. The <code><4</code> upper bound guards against an unknown future Python 4 breaking the world.	11

B.2 Runtime core frameworks

The packages the node is built on — the ones whose abstractions shape the architecture.

Package	Version	What it is / why Hathor uses it	Chapter
<code>twisted</code>	<code>~24.7.0</code>	The asynchronous networking engine (:57). Its reactor is the single event loop the entire node runs on; its Deferred , Protocol , and Factory abstractions underpin every connection. The load-bearing dependency of the whole codebase.	16
<code>autobahn</code>	<code>~24.4.2</code>	A WebSocket implementation that runs on Twisted (:58). Provides the <code>WebSocketServerFactory / Protocol</code> classes behind both the admin streaming surface and the event-queue stream.	36
<code>zope-interface</code>	<code>=8.2</code>	A formal interface system (:87) that Twisted relies on to declare contracts (e.g. the reactor interfaces). Pinned exactly because interface changes ripple widely. The reactor abstraction intersects three of its interfaces into one typed <code>ReactorProtocol</code> .	16, 23
<code>pydantic</code>	<code>^2.0</code>	Run-time data validation using type hints (:80). Validates everything crossing the node's boundary — settings, CLI args, API bodies — turning untyped external input into trusted typed objects. The run-time complement to mypy's static checks.	18
<code>typing-extensions</code>	<code>~4.12.2</code>	Back-ports of newer <code>typing</code> features to the supported Python versions (:82). Lets the code use modern type-system constructs uniformly.	5

B.3 Storage & serialization

Package	Version	What it is / why Hathon uses it	Chapter
<code>rocksdb</code>	<code>{git = ...}</code>	The embedded key-value store that persists the entire ledger (:74). Pulled from Hathon's own fork of <code>python-rocksdb</code> (github.com/hathornetwork/python-rocksdb) rather than PyPI, because the node needs a specific build of the C++ bindings. Chosen over a database server (MongoDB/SQL) precisely because it is embedded, ordered, and accessed by key.	27
<code>cffi</code>	<code>=1.17.1</code>	The C Foreign Function Interface (:86) — the bridge that lets Python call the compiled C/C++ code inside RocksDB and the crypto libraries. The mechanism behind "push the slow parts into C" from Chapter 11.	11, 27
<code>pyyaml</code>	<code>^6.0.1</code>	A YAML parser (:81). Reads the network settings profiles (<code>mainnet / testnet</code>) that define which network a node joins.	22
<code>sortedcontainers</code>	<code>~2.4.0</code>	Pure-Python sorted list/dict/set types (:72). Used where the node needs collections kept in order efficiently — e.g. timestamp-ordered structures in indexes and the mempool.	28
<code>base58</code>	<code>~2.1.1</code>	Base58 encoding (:59) — the alphabet (no <code>0</code> , <code>o</code> , <code>I</code> , <code>l</code>) used to render addresses as human-safe strings. The final step of the address pipeline.	40

B.4 Networking & APIs

Package	Version	What it is / why Hathon uses it	Chapter
<code>aiohttp</code>	<code>~3.10.3</code>	An async HTTP client (:75). Used for outbound HTTP — notably peer discovery (fetching bootstrap peer lists).	34

Package	Version	What it is / why Hathor uses it	Chapter
<code>requests</code>	<code>=2.32.3</code>	The classic synchronous HTTP client (:69), pinned exactly. Used by tooling and tests for straightforward request/response calls where the async machinery is unnecessary.	—
<code>idna</code>	<code>~3.4</code>	Internationalized Domain Names handling (:76) — correct encoding of non-ASCII hostnames, needed for TLS and DNS-based peer discovery.	34
<code>service_identity</code>	<code>~21.1.0</code>	Verifies that a TLS certificate actually matches the host you connected to (:70). Closes a class of man-in-the-middle attacks on encrypted peer connections.	34
<code>multidict</code>	<code>=6.7.0</code>	A dictionary allowing repeated keys (:84), pinned exactly. A dependency of the HTTP stack (headers can repeat); used where multi-valued mappings are needed.	—

B.5 Cryptography

The "never roll your own crypto" stack — standard, audited libraries wrapped thinly.

Package	Version	What it is / why Hathor uses it	Chapter
<code>cryptography</code>	<code>~42.0.5</code>	The primary cryptography library (:62) — ECDSA signing/verifying on the secp256k1 curve, key generation, hashing. The <code>Wallet</code> signs transactions through it. Wrapped by <code>hathor/crypto/</code> .	40
<code>pyopenssl</code>	<code>=24.2.1</code>	A Python wrapper over OpenSSL (:67), pinned exactly. Provides the TLS layer that encrypts peer-to-peer connections (Twisted uses it for transport security).	34, 40

Package	Version	What it is / why Hathor uses it	Chapter
<code>pycoin</code>	<code>~0.92.20230326</code>	A Bitcoin-style key utility library (:68). Provides BIP32 hierarchical-deterministic key derivation for the <code>HDWallet</code> ; the <code>hathor/pycoin/</code> wrapper registers Hathor's address version bytes so derived addresses are correct. (Note: the HD wallet signs via pycoin's own ECDSA — flagged in Chapter 40.)	40
<code>mnemonic</code>	<code>~0.20</code>	BIP39 mnemonic phrases (:65) — the word lists that turn a random seed into the 12/24 human-writable recovery words and back. Behind the <code>gen_hd_words</code> CLI.	40

B.6 Observability & logging

Package	Version	What it is / why Hathor uses it	Chapter
<code>structlog</code>	<code>~22.3.0</code>	Structured logging (:73) — records log entries as machine-readable key-value events rather than prose, so a long-running node's logs can be queried and aggregated. The same calls render human-readable in dev and JSON in production.	17
<code>prometheus_client</code>	<code>~0.15.0</code>	The Prometheus metrics library (:66). The node maps its counters/gauges (peer counts, tx/block rates, mempool size) through it; the exporter writes them to a <code>.prom</code> file scraped by an external collector.	42
<code>python-healthchecklib</code>	<code>^0.1.0</code>	A small health-check framework (:83). Powers the <code>/health</code> endpoint that tells an orchestrator (e.g. Kubernetes) whether the node is alive and ready.	42

Package	Version	What it is / why Hathor uses it	Chapter
<code>sentry-sdk</code>	<code>^1.5.11</code> (optional)	Error-tracking service client (:78). When enabled (the <code>sentry</code> extra), unhandled exceptions are forwarded to Sentry for alerting. Optional so the node runs fine without it — an example of the fallback pattern from Chapter 4.	17, 42
<code>structlog-sentry</code>	<code>^1.4.0</code> (optional)	The glue (:79) that pipes structlog events into Sentry. Part of the same optional <code>sentry</code> extra.	17

B.7 CLI, process & utilities

Package	Version	What it is / why Hathor uses it	Chapter
<code>configargparse</code>	<code>~1.7.1</code>	An <code>argparse</code> superclass (:61) letting one declared option be set from a command-line flag, an environment variable, or a config file. Gives the whole CLI an <code>HATHOR_*</code> env-var interface with one setting.	19
<code>colorama</code>	<code>~0.4.6</code>	Cross-platform coloured terminal output (:60). Makes the CLI's help text and the structured-log console renderer legible (and correct on Windows).	19, 21
<code>ipython</code>	<code>~8.7.0</code> (+kernel)	An enhanced interactive Python shell (:64). Powers <code>hathor-cli shell</code> — a REPL with the running node's objects loaded, invaluable for live exploration.	11, 21
<code>setproctitle</code>	<code>^1.3.3</code>	Sets the process's name as it appears in <code>ps / top</code> (:77), so operators can identify a running node at a glance.	—
<code>pexpect</code>	<code>~4.8.0</code>	Drives interactive subprocesses by expecting and responding to their output (:71). Used in tooling/tests that automate command-line interactions.	—
<code>graphviz</code>	<code>~0.20.1</code>	Python bindings to the Graphviz graph-drawing tool (:63). Renders the DAG to a visual diagram — a debugging/inspection aid for a graph-shaped ledger.	8

Package	Version	What it is / why Hathor uses it	Chapter
packaging	=26.0	Version-parsing and comparison utilities (: 88), pinned exactly. Used where the code reasons about version strings (e.g. migrations, compatibility checks).	—
hathorlib	{path, develop}	Not third-party — a sibling library developed in-tree (: 85 , develop = true means edits are picked up live). Holds shared settings, genesis data, and transaction utilities used by both the full node and lighter clients. Most settings fields actually live here (Chapter 22).	22

B.8 Development & quality toolchain

These live in the `[tool.poetry.group.dev.dependencies]` group (`pyproject.toml:40`) — needed to develop and test the node, deliberately excluded from the production Docker image via `--only=main` (Chapter 15).

Package	Version	What it is / why Hathor uses it	Chapter
mypy	^1.19.1	The static type checker (: 43) — reads the type hints across the codebase and reports inconsistencies before the code runs. A required merge gate. Restricted to CPython via a marker.	5, 20
mypy-zope	^1.0.14	A mypy plugin (: 44) teaching the checker to understand <code>zope.interface</code> contracts (which Twisted uses) — without it, type-checking the networking code would be impossible.	5, 20
flake8	~7.1.1	The linter (: 41) — flags stylistic problems and likely bugs (unused imports, undefined names, over-long lines) by static inspection.	20
isort	~5.13.2 (+colors)	The import formatter (: 42) — sorts and groups imports into a canonical order, ending all debate over import ordering. Its line length is kept equal to flake8's.	20

Package	Version	What it is / why Hathor uses it	Chapter
<code>pytest</code>	<code>~8.3.2</code>	The test runner (:45) — discovers and executes the test suite, reports pass/fail.	20, 43
<code>pytest-cov</code>	<code>~5.0.0</code>	A pytest plugin (:46) measuring code coverage — which lines the tests actually execute. Some test targets enforce a coverage floor.	20
<code>pytest-xdist</code>	<code>~3.6.1</code>	A pytest plugin (:48) distributing tests across CPU cores (<code>-n auto</code>), keeping a large suite's feedback loop short.	20
<code>flaky</code>	<code>~3.8.1</code>	A pytest plugin (:47) that re-runs tests which fail intermittently, distinguishing genuine failures from timing-flakes — useful for a system with concurrency.	20, 43
<code>yamllint</code>	<code>~1.35.1</code>	A linter for YAML files (:49) — catches malformed settings profiles and config before they reach the node. Part of <code>make check</code> .	20, 22
<code>types-requests</code> , <code>types-pyopenssl</code> , <code>types-pyyaml</code>	pinned	Type-stub packages (:52 - :54) — they add type hints for third-party libraries that ship none, so mypy can check code that calls into <code>requests</code> , <code>pyOpenSSL</code> , and <code>PyYAML</code> .	5, 20

B.9 Build & packaging

Not in the dependency lists proper, but part of the manifest — the machinery that turns the source tree into an installable package.

Item	Value	What it is / why Hathon uses it	Chapter
build backend	<code>poetry.core.masonry.api</code>	The code that packages the project into a wheel/sdist (<code>pyproject.toml: 191</code>). Declared per the PEP 517/518 standard, so the project can be built by tooling other than Poetry. The Docker build runs <code>poetry build -f wheel</code> against exactly this.	13, 15
<code>[tool.poetry.scripts]</code>	<code>hathon-cli = 'hathon_cli.main:main'</code>	The console-script entry (<code>:38</code>) that creates the <code>hathon-cli</code> terminal command, wiring it to the <code>main()</code> function. The book's front door.	21
<code>[tool.poetry.extras]</code>	<code>sentry = [...]</code>	The optional-feature bundle (<code>:90</code>) — installing the <code>sentry</code> extra pulls in <code>sentry-sdk</code> + <code>structlog-sentry</code> together.	17, 42

Recap

Group	Count	The load-bearing ones
Runtime core	5	<code>twisted</code> , <code>pydantic</code>
Storage & serialization	5	<code>rocksdb</code> (custom fork), <code>pyyaml</code>
Networking & APIs	5	<code>aiohttp</code> , <code>service_identity</code>
Cryptography	4	<code>cryptography</code> , <code>pycoin</code>

Group	Count	The load-bearing ones
Observability	5	<code>structlog</code> , <code>prometheus_client</code>
CLI & utilities	8	<code>configargparse</code> , <code>ipython</code> , <code>hathorlib</code>
Dev & quality	12	<code>mypy</code> , <code>pytest</code>
Build & packaging	3	the Poetry backend

Every package in the node's `pyproject.toml` earns its place against the same test: it solves a problem the node would otherwise have to solve itself, badly. Twisted gives it concurrency, RocksDB gives it durable ordered storage, Pydantic and the type stubs give it boundary safety, the crypto libraries give it audited primitives no one should hand-roll, and the dev group gives it the discipline to stay correct as it grows. The manifest is short by the standards of a modern application — a deliberate choice, since every dependency is also a thing that can break, be compromised, or drift. With this annotated, the last appendix collects the quick-reference tables — column-family layouts, message types, command lists — that you reach for once the concepts are already in hand.

-
1. A manifest is the file listing a project's dependencies and metadata. For a Poetry project that is `pyproject.toml` (declared ranges) paired with `poetry.lock` (resolved exact versions). ↩
 2. An extra (optional dependency group) is a named bundle of packages a user can choose to install — here, `sentry`. Code guards their use so the node runs whether or not they are present. ↩
 3. A type stub is a file (or package, named `types-*`) providing type hints for a library that ships none, so a static checker like mypy can verify code that uses that library. ↩